```
      {N: input['--duebefore']};
  }
  if (input['<category>'] !== null) {
    if (params.FilterExpression === undefined) {
      params.FilterExpression = '';
    } else {
      params.FilterExpression += ' AND ';                    ⑥
    }
    params.FilterExpression += 'category = :category';
    params.ExpressionAttributeValues[':category'] =
      S: input['<category>']};
  }
  db.query(params, (err, data) => {                          ⑦
    if (err) {
      console.error('error', err);
    } else {
      console.log('tasks', data.Items.map(mapTaskItem));
      if (data.LastEvaluatedKey !== undefined) {
        console.log('more tasks available with --next=' +
          data.LastEvaluatedKey.tid.N);
      }
    }
  });
}
```

① Primary key query. The task table uses a partition and sort key. Only the partition key is defined in the query, so all tasks belonging to a user are returned.

② Query attributes must be passed this way.

③ Filtering uses no index; it's applied over all elements returned from the primary key query.

④ Filter attributes must be passed this way.

⑤ attribute_not_exists(due) is true when the attribute is missing (opposite of attribute_exists).

⑥ Multiple filters can be combined with logical operators.

⑦ Invokes the query operation on DynamoDB

As an example, you could use the following command to get a list of Emma's shopping tasks. The query will fetch all tasks by `uid` first and filter items based on the category attribute next:.

```
node index.js task-ls emma shopping
```

Two problems arise with the query approach: